

解题报告

姓名：白家辉

班级：计算机大类2508

学号：8208250831

日期：2026.4.12

总览

本次解题报告中，共完成 4 道题目。

题目名称	难度	知识点
P1047 校门外的树	入门	布尔数组
P3156 询问学号	入门	数组、顺序存储
P1996 约瑟夫问题	普及	队列、模拟
P1160 队列安排	普及/提高-	双向链表、链表模拟

（一）P1047 校门外的树

题目信息

题目链接：<https://www.luogu.com.cn/problem/P1047>

知识点：布尔数组

题目简述

某校大门外长度为 l 的马路上有一排树，每两棵相邻的树之间的间隔都是 1 米。我们可以把马路看成一个数轴，马路的一端在数轴 0 的位置，另一端在 l 的位置；数轴上的每个整数点，即

$0, 1, 2, \dots, l$ ，都种有一棵树。现在要把这些区域中的树（包括区域端点处的两棵树）移走。你的任务是计算将这些树都移走后，马路上还有多少棵树。

对于 20% 的数据，保证区域之间没有重合的部分。

对于 100% 的数据，保证 $1 \leq l \leq 10^4$ ， $1 \leq m \leq 100$ ， $0 \leq u \leq v \leq l$

1.00s, 125MB

题目分析

1. 读题

马路长度为 1，从 0 到 1 每个整数点共种植 $l+1$ 棵树。现在给出 m 个待移除区间，要求移除区间内所有树（包含端点），区间可能存在重叠，计算移除完成后剩余的树木数量，核心需要处理重叠部分的重复计数问题。

2. 性质观察

本题可以通过转换思路简化问题：只要把被移除的位置标记出来，最后统计没有被标记的位置数量即可。由于树只种在整数点上，区间端点也都是整数，所以可以直接按位置开数组并逐点处理每个区间。结合当前代码来看，即使某个位置被重复覆盖，也只是让对应计数继续增加，不会影响最后“是否剩余”的判断。

3. 程序设计思路

当前实现采用的是顺序模拟加数组计数：先申请一个长度为 $l+1$ 的整型数组，默认所有位置都表示“尚未被移除”；之后依次读入每个区间 $[a, b]$ ，把区间内每个位置的计数加一；最后再扫一遍数组，统计计数仍为 0 的位置个数，也就是剩余树木数量。虽然这里用的是整型数组而不是布尔数组，但实现思路本质一致，写起来更直接。

4. 数据结构与算法选择

当前代码选择的是一维整型数组。数组下标直接对应马路上的位置：

1. $arr[i]=0$ 表示该位置的树还没有被任何区间覆盖；
2. 扫描区间时对 $arr[i]$ 执行加一，表示该位置已被移除；
3. 最后统计 $arr[i]==0$ 的位置数量。

这种做法不需要额外去重逻辑，数组本身就把覆盖情况记录下来了。

5. 时空复杂度分析

- 时间复杂度： $O(l \times m)$ ，根据数据范围 $l \leq 10^4, m \leq 100$ ，总操作次数不超过 10^6 ，完全符合时间要求。
- 空间复杂度： $O(l)$ ，仅需要一个长度为 $l+1$ 的布尔数组，空间占用极低，满足题目限制。

代码实现

洛谷 / 评测记录 / 评测详情

R281438163 记录详情

编程语言C O2 | 代码长度421B | 用时30ms | 内存1.05MB

测试点信息源代码

测试点信息

#1 AC 3ms/792.00KB	#2 AC 3ms/788.00KB	#3 AC 3ms/820.00KB	#4 AC 3ms/1.05MB	#5 AC 3ms/816.00KB	#6 AC 3ms/816.00KB	#7 AC 3ms/1.04MB
#8 AC 3ms/812.00KB	#9 AC 3ms/796.00KB	#10 AC 3ms/728.00KB				

jsj0708Baijiahui

所属题目P1047 [NOIP 2005 普及组] 校门...

评测状态Accepted

评测分数100

提交时间2026-06-11 13:51:58

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int l=0;
    int n=0;
    int sum=0;
    scanf("%d %d",&l,&n);
    int* arr=(int*)malloc((l+1)*sizeof(int));
    for(int i=0;i<n;i++)
    {
        int a,b;
        scanf("%d %d",&a,&b);
        for(int i=a;i<=b;i++)
            arr[i]++;
    }
    for(int i=0;i<=l;i++)
    {
        if(arr[i]==0)
        {
            sum++;
        }
    }
    printf("%d",sum);
}
```

(二) P3156 询问学号

题目信息

题目链接：<https://www.luogu.com.cn/problem/P3156>

知识点：数组、顺序存储

题目简述

按进入教室的顺序给出 n 名同学的学号，老师会进行 m 次询问。每次询问给出一个位置 i ，要求输出第 i 个进入教室的同学的学号。

题目中同学进入教室的顺序在输入完成后不再变化，因此本质上是在一段静态序列上做多次按位置查询。

对于 100% 的数据， $n \leq 2 \times 10^6$ ，学号在 1 到 10^9 之间，询问次数 $m \leq 10^5$ 。

1.00s, 128MB。

题目分析

1. 读题

题目只做两件事：先读入所有同学的学号顺序，再多次询问“第 i 个进入教室的人是谁”。这里没有交换、删除、插入等修改操作，因此进入顺序就是一份固定不变的数据。

2. 性质观察

第 i 个进入教室的同学的学号，实际上就是输入序列中的第 i 个数。因此只要把输入顺序原样保存下来，之后所有询问都可以直接按位置访问。结合当前代码来看，作者采用的是从 1 开始存数组下标，这样查询时可以直接用题目给出的编号访问 `arr[temp]`，避免了额外做下标转换。

3. 程序设计思路

先申请一个长度为 $m+1$ 的数组，并按照输入顺序把学号存到 `arr[1]` 到 `arr[m]` 中。随后再读入每个询问位置，直接输出对应下标处保存的学号即可。整个过程不需要排序，也不需要额外预处理，核心就是一次存储、多次直接访问。

4. 数据结构与算法选择

最合适的数据结构是顺序表，也就是数组。因为：

1. 输入本身就是一段线性序列；
2. 题目只需要通过位置访问元素；
3. 数组支持按下标 $O(1)$ 访问，能够高效回答大量查询。

当前实现还特意把数组开成了从 1 开始使用的形式，因此和题目中的“第几个同学”能够一一对应，代码比较直观。

5. 时空复杂度分析

- 时间复杂度： $O(n + m)$ 。读入学号需要 $O(n)$ ，回答全部询问需要 $O(m)$ 。

- 空间复杂度： $O(n)$ 。只需要一个长度为 n 的数组保存所有学号。

在题目给定的数据范围下，这一做法完全可行。

代码实现

洛谷 / 评测记录 / 评测详情

R281440302 记录详情

编程语言: C O2 | 代码长度: 362B | 用时: 509ms | 内存: 7.28MB

测试点信息 | 源代码

测试点信息

#1	#2	#3	#4	#5
AC	AC	AC	AC	AC
105ms/5.59MB	97ms/4.82MB	135ms/7.28MB	106ms/5.21MB	66ms/3.13MB

jsj0708Baijiahui

所属题目: P3156 【深基15.例1】询问学号

评测状态: Accepted

评测分数: 100

提交时间: 2026-06-11 14:24:27

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int m,n;
    scanf("%d %d",&m,&n);
    int* arr=(int*)malloc((m+1)*sizeof(int));
    for(int i=1;i<=m;i++)
    {
        int temp=0;
        scanf(" %d",&temp);
        arr[i]=temp;
    }
    for(int i=0;i<n;i++)
    {
        int temp=0;
        scanf(" %d",&temp);
        printf("%d\n",arr[temp]);
    }
}
```

(三) P1996 约瑟夫问题

题目信息

题目链接：<https://www.luogu.com.cn/problem/P1996>

知识点：队列、模拟

题目简述

n 个人围成一圈，从第一个人开始报数，数到 m 的人出圈，然后由下一个人重新从 1 开始报数。如此反复，直到所有人都出圈为止。

题目要求输出所有人依次出圈的编号顺序。需要特别注意，本题要求的是“全部出圈”的过程，而不是只求最后剩下的那个人。

对于 100% 的数据， $1 \leq n, m \leq 100$ 。

1.00s, 128MB。

题目分析

1. 读题

题目描述的是一个典型的循环报数过程。每轮都会从当前的人开始继续数，报到 m 的人离开队伍，下一位再从 1 重新开始。最终要输出完整的出圈顺序。

2. 性质观察

所谓“围成一圈”，本质上就是编号在一个环上不断循环。结合当前代码来看，并没有真的开一个队列反复进出，而是用数组记录某个人是否还在圈中：1 表示未出圈，0 表示已出圈。之后用下标不断循环扫描整个数组，并只对仍然存活的人计数，这样同样能够完成环形报数模拟。

3. 程序设计思路

当前实现思路是：

1. 先把长度为 n 的数组全部置为 1，表示每个人最初都还在圈里；

2. 用 `idx` 表示当前扫描到的位置，并通过 `idx=(idx+1)%n` 实现环形移动；
3. 只有 `arr[idx]==1` 时才参与报数；
4. 当计数器达到 `m` 时，就输出该位置对应的编号、把该位置改为 `0`，并把计数器清零；
5. 当已经出圈的人数达到 `n` 时结束。

虽然写法和标准队列模拟不同，但本质上仍然是在做约瑟夫过程的顺序模拟。

4. 数据结构与算法选择

当前代码更贴近“环形数组 + 存活标记”的做法。由于 `n, m` 范围很小，这种直接循环扫描的方法完全可行，实现上也比较直观，不需要真的维护一个动态队列。

5. 时空复杂度分析

- 时间复杂度： $O(nm)$ 。每淘汰一个人最多经历一轮长度为 m 的报数过程。
- 空间复杂度： $O(n)$ 。队列中最多同时存放 n 个人。

由于题目规模只有 100 级别，直接模拟可以稳定通过。

代码实现

洛谷 / 评测记录 / 评测详情

R281440388 记录详情

编程语言	代码长度	用时	内存
C++	504B	12ms	1.08MB

测试点信息

源代码

测试点信息

#1	#2	#3	#4
AC	AC	AC	AC
3ms/832.00KB	3ms/1.08MB	3ms/816.00KB	3ms/812.00KB

jsj0708Baijiahui

所属题目	P1996 约瑟夫问题
评测状态	Accepted
评测分数	100
提交时间	2026-06-11 14:25:21


```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int n,m;
    scanf("%d %d",&n,&m);
    int* arr=(int*)malloc(n*sizeof(int));
    int idx=0;
    for(int i=0;i<n;i++)
    {
        arr[i]=1;
    }
    int num=0;
    int count=0;
    while(num<n)
    {
        if(arr[idx]==1)
        {
            count++;
        }
        if(count==m)
        {
            printf("%d ",idx+1);
            num++;
            count=0;
            arr[idx]=0;
        }
        idx=(idx+1)%n;
    }
    free(arr);
}
```

(四) P1160 队列安排

题目信息

题目链接：<https://www.luogu.com.cn/problem/P1160>

知识点：双向链表、链表模拟

题目简述

先将 1 号同学放入队列。之后对于每个 $i(2 \leq i \leq N)$ ，给出一名已经在队列中的同学 k 和一个方向标记 p ，表示把 i 插入到 k 的左边或右边。

在所有插入完成后，再执行 M 次删除操作。每次给出一个编号 x ，表示将该同学从队列中删去；如果该同学已经不在队列中，则忽略这次删除。

最终要求输出队列从左到右的所有同学编号。对于 100% 的数据， $1 < M \leq N \leq 10^5$ 。

1.00s, 128MB。

题目分析

1. 读题

题目的操作分成两类：一类是在某个已知同学的左边或右边插入新同学，另一类是按编号删除某位同学。最后输出整个队列的排列结果。显然，这是一道动态维护线性序列的问题。

2. 性质观察

如果使用普通数组来维护整个队列，那么每次在中间插入或删除元素时，都需要移动后面大量元素。在 N 达到 10^5 时，这种做法的总复杂度会非常高。结合当前代码来看，作者正是利用“编号天然连续”这一点，直接用 `pre` 和 `aft` 两个数组维护每个人左边和右边是谁，从而避免整体搬移元素。

3. 程序设计思路

可以用数组模拟双向链表：

1. `L[i]` 表示编号 i 左边的同学；
2. `R[i]` 表示编号 i 右边的同学；
3. 再用一个标记数组记录某个同学当前是否还在队列中，避免重复删除。

在当前实现中，这三个数组对应为 `pre`、`aft` 和 `del`。插入时根据 `p` 的取值修改相邻结点关系；删除时若 `del[temp]==1`，就把该结点两侧直接连起来；最后再扫描一遍找到 `pre[i]==0` 的队首，从这里沿着 `aft` 指针一路输出最终结果。

4. 数据结构与算法选择

本题最适合使用双向链表。又因为同学编号天然连续的 $1 \sim N$ ，所以直接使用数组模拟双向链表会比动态开结点更加方便。当前代码还额外使用了 `del` 数组来处理“重复删除无效”这一要求，因此整体实现和题目操作是严格对应的。

5. 时空复杂度分析

- 时间复杂度： $O(N + M)$ 。每次插入、删除都是 $O(1)$ ，最后遍历输出一次队列也是线性的。
- 空间复杂度： $O(N)$ 。需要若干个长度为 N 的数组维护左右邻接关系和删除标记。

这一复杂度能够满足 10^5 级别数据的时限要求。

代码实现

洛谷 / 评测记录 / 评测详情

R281440463 记录详情

编程语言C++2代码长度1.03KB用时74ms内存1.67MB

测试点信息源代码

测试点信息

#1AC4ms/1.52MB

#2AC4ms/1.67MB

#3AC25ms/1.59MB

#4AC21ms/1.54MB

#5AC20ms/1.57MB

jsj0708Baijiahui

所属题目P1160 队列安排

评测状态Accepted

评测分数100

提交时间2026-06-11 14:26:25

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#define max 100010
int main()
{
    int pre[max] = { 0 };
    int aft[max] = { 0 };
    int del[max] = { 0 };
    int n = 0;
    scanf("%d", &n);
    for (int i = 1; i <= n; i++)
    {
        del[i] = 1;
    }
    for (int i = 0; i < n - 1; i++)
    {
        int k = 0;
        int p = 0;
        scanf("%d %d", &k, &p);
        if (p == 0)
        {
            int oldpre = pre[k];
            pre[k] = i + 2;
            pre[i + 2] = oldpre;
            aft[i + 2] = k;
            aft[oldpre] = i + 2;
        }
        else
        {
            int oldaft = aft[k];
            aft[k] = i + 2;
            aft[i + 2] = oldaft;
            pre[i + 2] = k;
            pre[oldaft] = i + 2;
        }
    }
    int m = 0;
    scanf("%d", &m);
    for (int i = 0; i < m; i++)
    {
        int temp = 0;
        scanf(" %d", &temp);
    }
}

```

```

        if (del[temp] == 1)
        {
            int temp1 = pre[temp];
            int temp2 = aft[temp];
            aft[temp1] = temp2;
            pre[temp2] = temp1;
            del[temp] = 0;
        }
    }
    int start = 0;
    for (int i = 1; i <= n; i++)
    {
        if (pre[i] == 0)
        {
            start = i;
            break;
        }
    }
    while (start != 0)
    {
        if (del[start] == 1)
        {
            printf("%d ", start);
        }
        start = aft[start];
    }
    return 0;
}

```